OntarioTech UNIVERSITY

Big Data System Design (MBAI 5110G)

Winter-2026

Week 3: NoSQL Databases

Lectured by:

Dilli P. Sharma, PhD

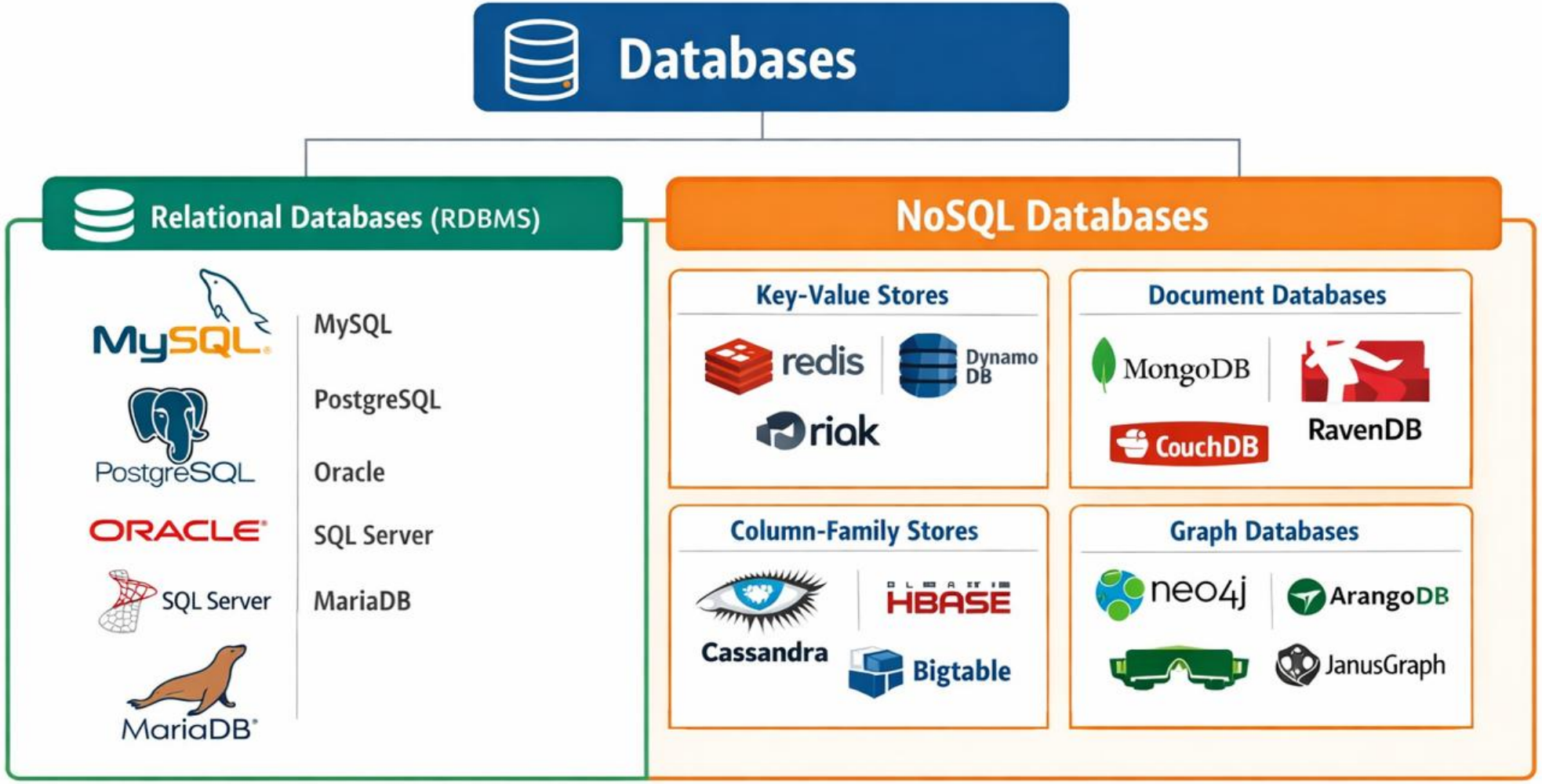
Outline

- Introduction to No SQL Databases
- CAP Theorem
- ACID Vs BASE Properties
- RDBMS vs NoSQL Databases
- Features of NoSQL Databases
- NoSQL Database Types and Use-cases

Introduction to NoSQL (Not Only SQL) Database

- **Big Data requires new storage solutions** because:
  - Data **volume, variety, and velocity** exceed the capacity of **traditional RDBMS.**
  - **NoSQL** databases solve the **scale and flexibility** challenges of Big Data storage
- What is a NoSQL database?
  - A class of **non-relational databases** designed for **large, distributed data sets**.
  - It supports flexible data models

Popular Database Systems



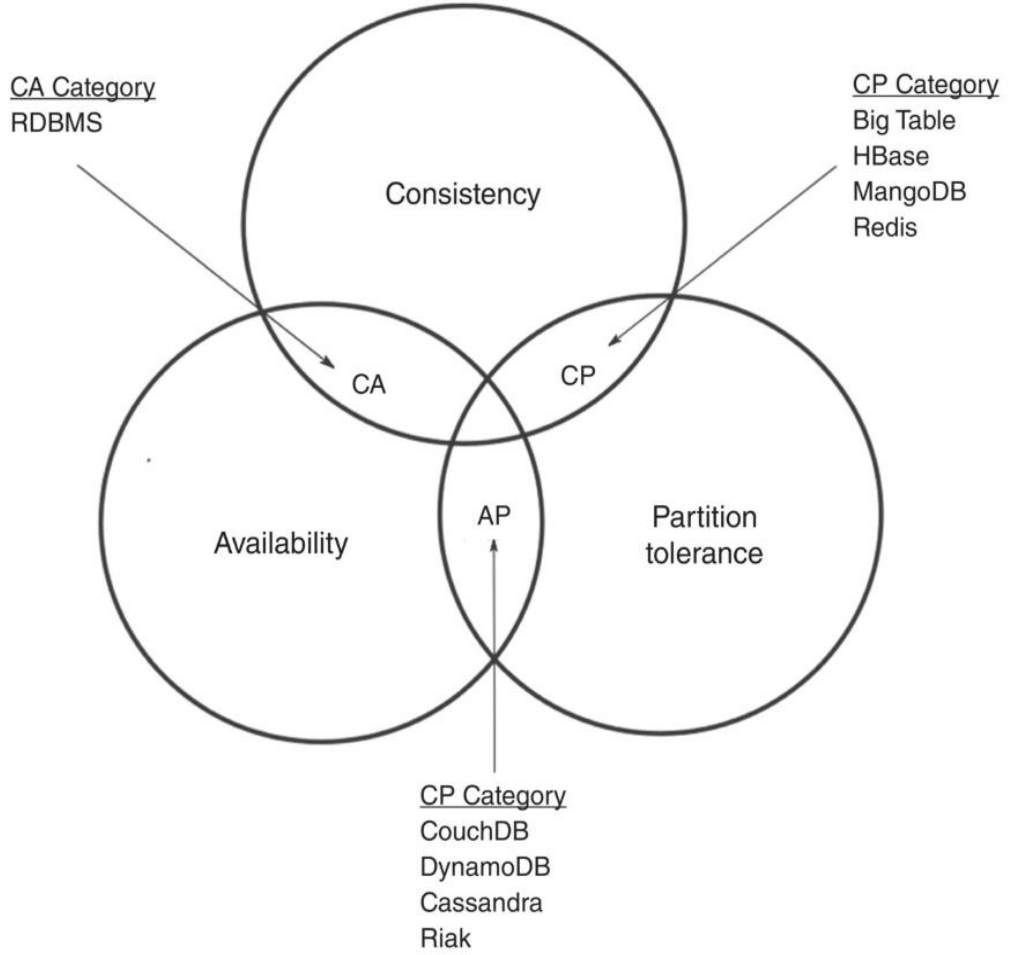
Why NoSQL?

- **RDBMS** was the primary solution for all database needs
- In recent years, **massive volumes of data** have been generated from multiple sources, and most of this data is:
  - Unstructured
  - Semi-structured
  - Rapidly changing
- **Traditional RDBMS** systems:
  - Support only **structured data**
  - Require **predefined schemas**
  - Struggle with **scalability and flexibility**

- **NoSQL Database**:
  - Handles **structured, semi-structured, and unstructured data.**
  - **Scales horizo**ntally across many machines easily
  - Supports **fast ingestion and retrieval of large data volumes**

CAP Theorem

- It is a fundamental concept in **distributed database systems**
- The **CAP** theorem says that a distributed system can deliver only two of three desired characteristics:
  - ***Consistency, Availability,*** and ***Partition tolerance***
  - **C**: All nodes return the **same data** for a read request
  - **A**: Every read or write request receives a **response**
  - **P**: System continues to operate despite **network failures**
- Explains trade-offs in NoSQL and Big Data systems



Properties of a system following the CAP theorem

- **Relational databases** achieve **CA**
- **NoSQL databases** are designed to achieve either **CP or AP**
- **NoSQL databases** exhibit partition tolerance at the cost of sacrificing either consistency or availability

ACID vs BASE Properties

- **ACID (RDBMS):**
  - **Atomicity (A):** Transactions are **all or nothing**
  - **Consistency (C):** Database remains in a **valid state** after a transaction
  - **Isolation (I)**: Concurrent transactions do **not interfere** with each other
  - **Durability (D)**: Completed transactions are **permanent**, even after system crashes

- **BASE (NoSQL):**
  - **Basically available**: System **always responds**, even during failures. Focus on availability over strict correctness
  - **Soft state**: Data may be **temporarily inconsistent** across nodes
  - **Eventual consistency**: Database will **eventually become consistent** across all nodes

Feature	ACID	BASE
Guarantee	Strong consistency	Eventual consistency
Availability	May sacrifice availability	High availability
Transaction Model	Synchronous	Asynchronous / distributed
Scalability	Vertical scaling	Horizontal scaling
Use case	Banking, financial transactions	Social networks, IoT, Big Data, real-time apps

Which one (ACID or BASE) best fits a Big Data System?

Modern Big Data systems often **favor BASE** to handle high **volume, velocity, and variety of data.**

RDBMS vs NoSQL

Data Structure	• **Relational databases** use a fixed schema with tables, rows, and columns. • **NoSQL databases** are schema-flexible. Data can be stored as JSON documents, key-value pairs, or graphs, allowing each entry to look different.
Scalability	• **Relational databases** are typically scaled vertically- adding more power to a single server (CPU, RAM, etc.). • **NoSQL databases** are designed for horizontal scaling—distributing data across many servers to handle high traffic and large datasets.
Query Language	• **Relational Database** uses Structured Query Language (SQL) • **NoSQL Databases** have variants that depend on the type
Transactions	• **Relational data**base provides strong ACID properties • **NoSQL database supports** BASE or CAP properties

Relational (RDBMS) Vs NoSQL (MongoDB)

Relational

ID	first_name	last_name	cell	city	year_of_birth	location_x	location_y
1	'Mary'	'Jones'	'516-555-2048'	'Long Island'	1986	'-73.9876'	'40.7574'

ID	user_id	profession
10	1	'Developer'
11	1	'Engineer'

ID	user_id	name	version
20	1	'MyApp'	1.0.4
21	1	'DocFinder'	2.5.7

ID	user_id	make	year
30	1	'Bentley'	1973
31	1	'Rolls Royce'	1965

MongoDB

```
{
first_name: "Mary",
last_name: "Jones",
cell: "516-555-2048",
city: "Long Island",
year_of_birth: 1986,
location: {
        type: "Point",
        coordinates: [-73.9876, 40.7574]
},
profession: ["Developer", "Engineer"],
apps: [
 { name: "MyApp",
   version: 1.0.4 },
 { name: "DocFinder",
   version: 2.5.7 }
 ],
cars: [
   { make: "Bentley",
     year: 1973 },
   { make: "Rolls Royce",
     year: 1965 }
 ]
}
```

Querying in SQL and NoSQL (MongoDB) Database

SQL	MongoDB

`CREATE TABLE users (`
`user_id VARCHAR(20) NOT NULL,`
`age INTEGER NOT NULL,`
`status VARCHAR(10));`
	Not Required

`INSERT INTO users(user_id, age, status)`
`VALUES ('bcd001', 45,"A");`
	`db.users.insert({`
	`  user_id: "bcd001",`
	`  age: 45,`
	`  status: "A"`
	`})`

`SELECT *` `FROM users;`	`db.users.find()`

`UPDATE users`
`SET status = 'C'`
`WHERE age > 25;`
	`db.users.update(`
	`  { age: { $gt: 25 } },`
	`  { $set: { status: "C" } },`
	`  { multi: true }`
	`)`

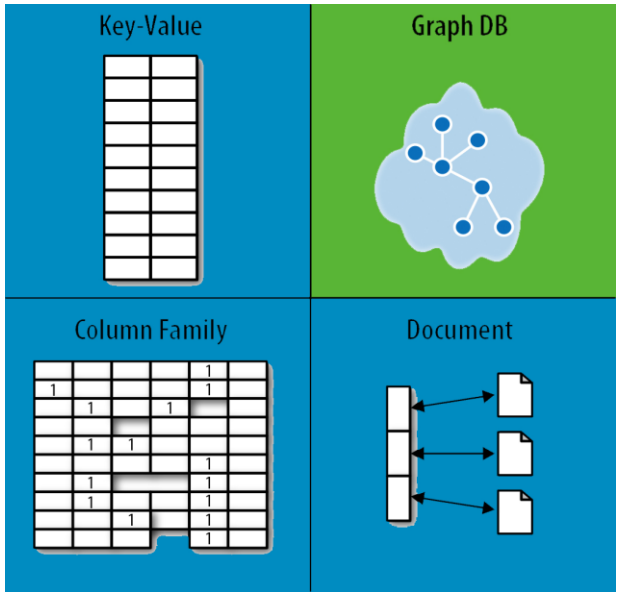
Features of NoSQL Databases

- Flexibility (**schemaless**)
- Scalability (**horizontal**)
- Reliability (**distributed architecture**)
- Cost-efficiency (**commodity hardware**)
- Non-Relational
- High performance for **massive data volumes**

These features make NoSQL ideal for **modern Big Data applications!**

NoSQL Database Types

- **Sorted Column Store:**
  - Optimized for queries over large datasets, and stores columns of data together, instead of rows
  - Examples: *Cassandra, HBase, Google Bigtable*
- **Document databases:**
  - Stores data as **documents (JSON, BSON, XML)**
  - **Exemples:** *MongoDB, CouchDB*
- **Key-Value Store :**
  - Is a simplest NoSQL databases.
  - Stores data as **key-value pairs**
  - Examples: *Redis, Riak, DynamoDB*
- **Graph Databases :**
  - Data is stored as **nodes and edges**
  - Excellent for **relationships, networks, and social graphs**
  - Examples: *Neo4j, ArangoDB, JanusGraph*



Column-Store Database

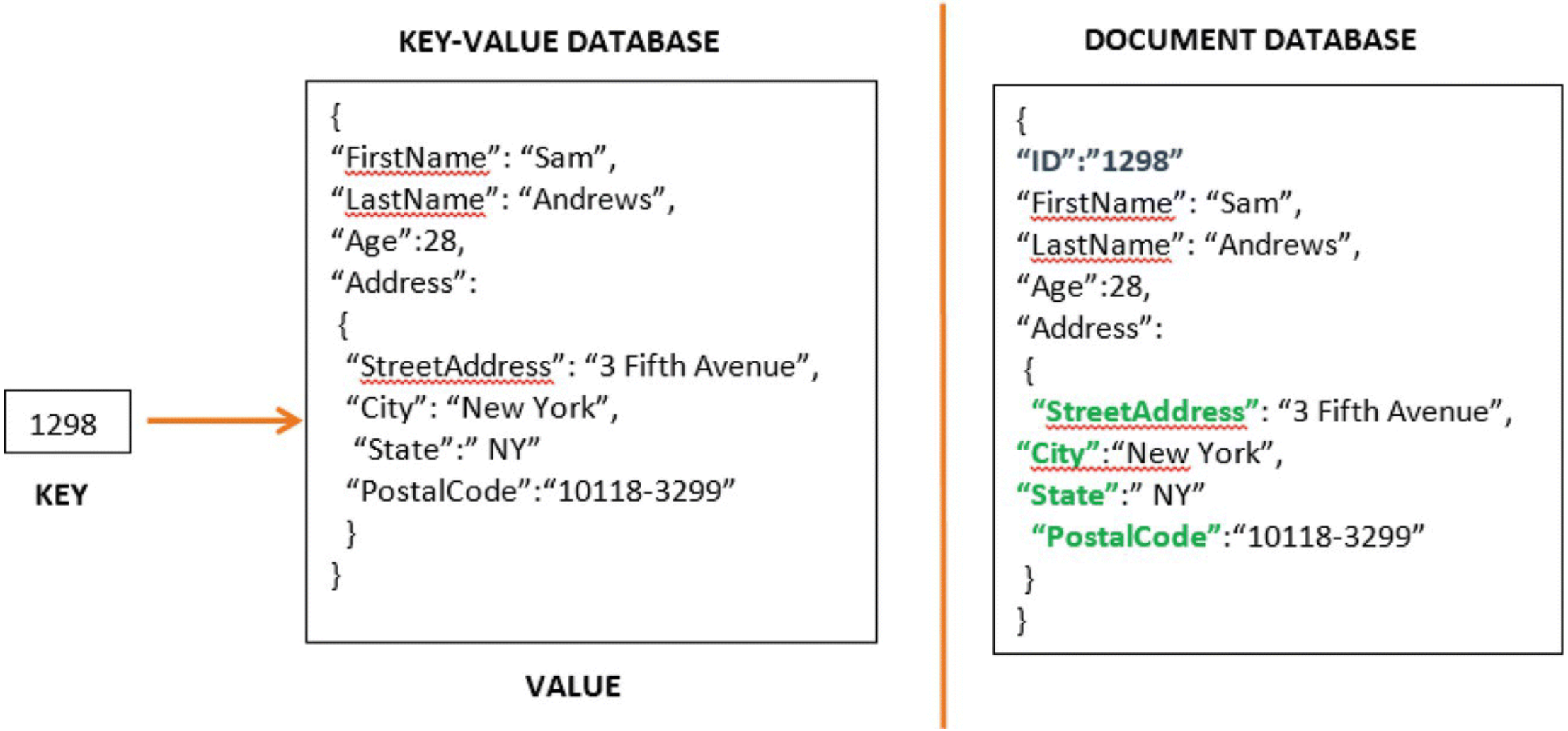
Employee_Id	Name	Salary	City	Pin_code
3623	Tony	$6000	Huntsville	35801
3636	Sam	$5000	Anchorage	99501
3967	Williams	$3000	Phoenix	85001
3987	Andrews	$2000	Little Rock	72201

Row store database

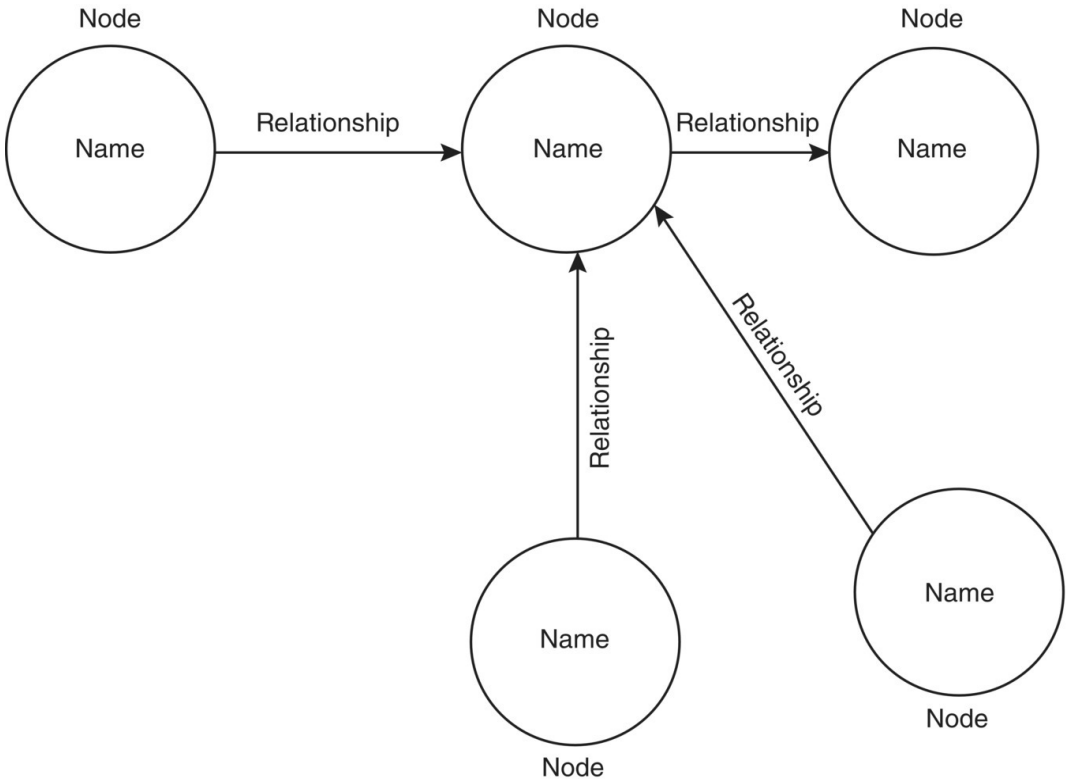
Employee_Id	Name	Salary	City	Pincode
3623	Tony	$6000	Huntsville	35801
3636	Sam	$5000	Anchorage	99501

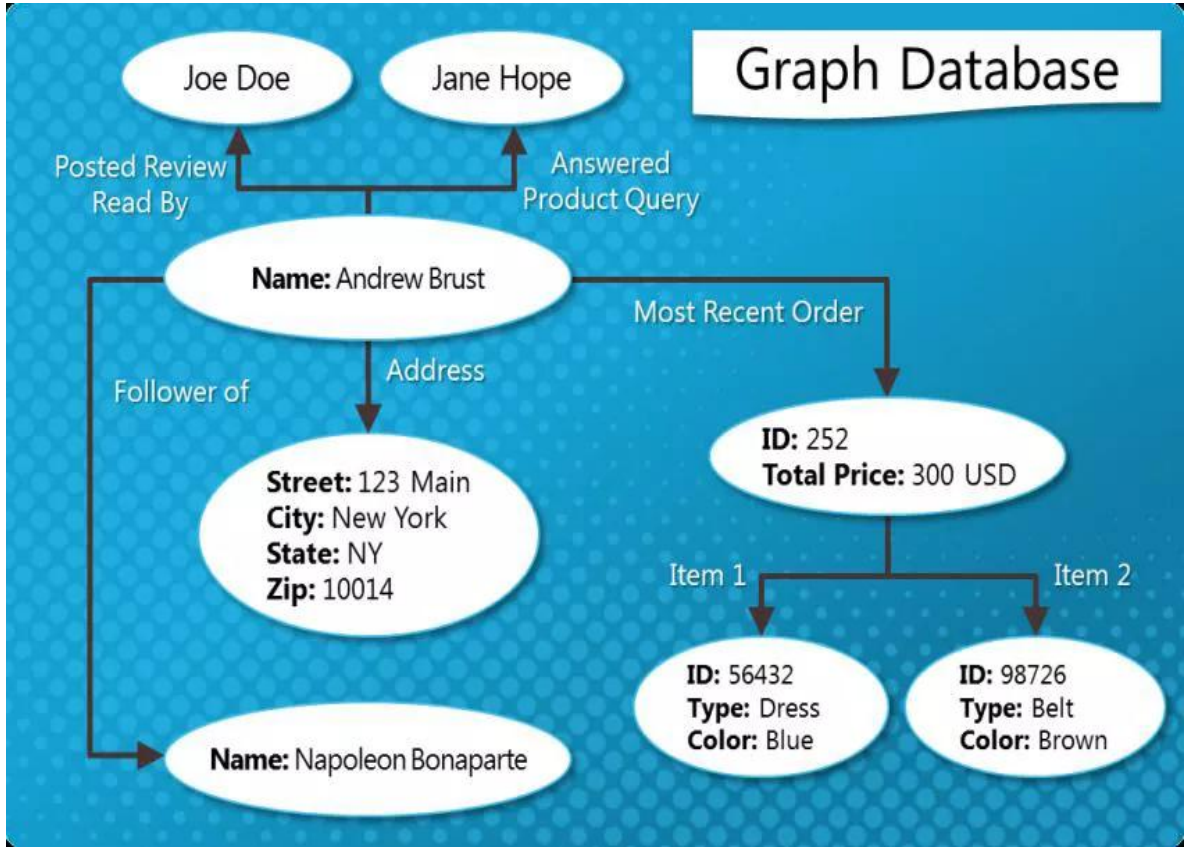
Column store database

Key-Vale and Document Database



Graph Database





NoSQL Database Types and their Use-Cases

Document Databases

Store data as JSON or BSON documents, ideal for semi-structured data and flexible schemas. These are great for content management, catalogues, and applications with fast-evolving data models.

- Popular tools: MongoDB, CouchDB
- Use case: Product data, blogs, user profiles

Key-Value Stores

These are the simplest NoSQL databases: a giant hash table where each key maps to a value. Extremely fast and highly scalable, often used in caching and session management.

- Popular tools: Redis, Riak, DynamoDB (in key-value mode)
- Use case: Session storage, user preferences, shopping carts

Wide-Column Stores

These are utilised in distributed systems at a large scale and offer rows and columns in a table with some type of difference in the columns associated with each row, not all the columns. Like a flexible spreadsheet.

- Popular tools: Apache Cassandra, HBase
- Use case: Time-series data, IoT, analytics workloads

Graph Databases

These focus on relationships between data. Nodes represent entities, and edges represent relationships—ideal for social networks, fraud detection, and recommendation engines.

- Popular tools: Neo4j, ArangoDB, Amazon Neptune
- Use case: Friend-of-a-friend queries, supply chain management

Reference/textbook

- Chapter 3-NoSQL Databases (Textbook): Big Data: Concepts, Technology, and Architecture, Balamurugan Balusamy, Nandhini Abirami R, Seifedine Kadry, Amir H. Gandomi, Wiley, 2021

Next Session: Big Data Processing and Management in Cloud

Q/A

Thank you!